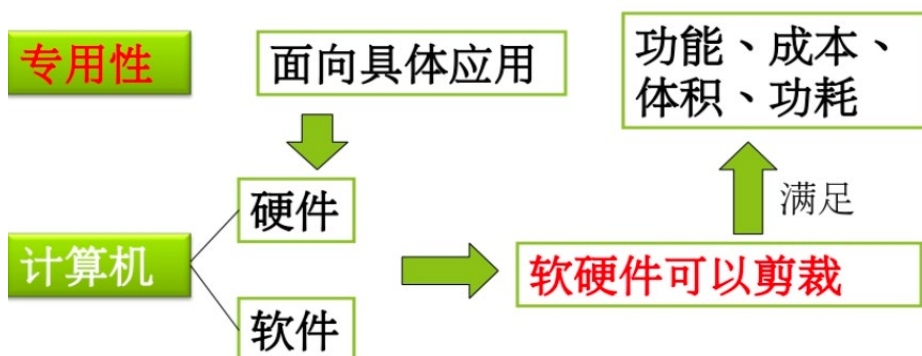


嵌入式复习资料

第一章 嵌入式系统概论

1. 嵌入式系统的定义

- ①IEEE 定义：嵌入式系统是“控制、监视或者辅助设备、机器和车间运行的装置。
- ②嵌入式系统是以应用为中心，以计算机技术为基础，采用可剪裁软硬件，适用于对功能、可靠性、成本、体积、功耗等有严格要求的专用计算机系统。



2. 嵌入式系统发展的三个阶段

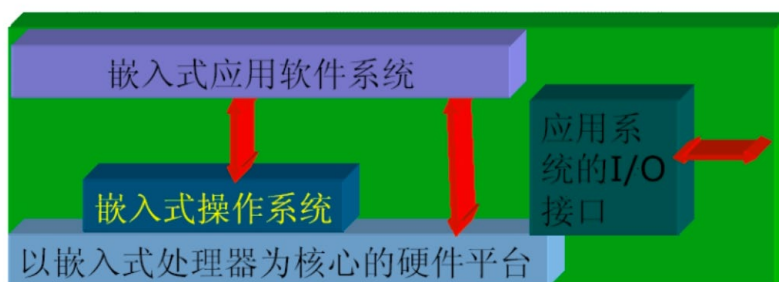
- ①嵌入式系统的出现和兴起 (1960-1970)
- ②嵌入式系统走向繁荣，软件、硬件日臻完善(1971-1989)
- ③嵌入式系统应用走向纵深化发展 (1990-至今)

3. 嵌入式构成

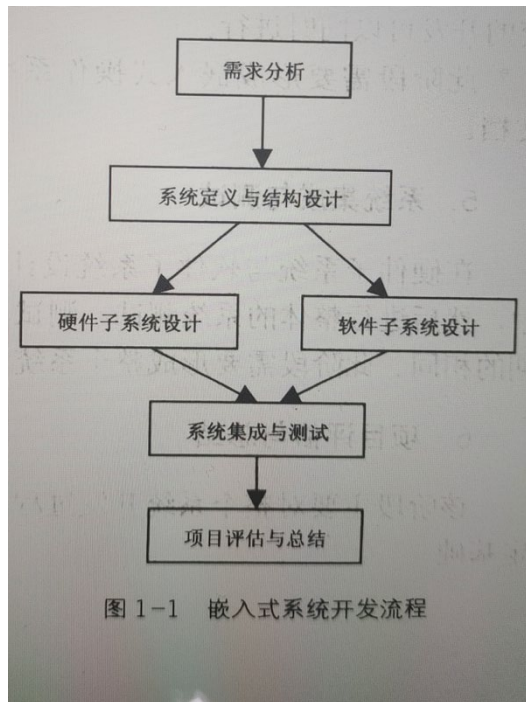
嵌入式系统一般由嵌入式微处理器、外围硬件设备、嵌入式操作系统(可选)，以及用户的应用软件系统等四个部分组成。

硬件包括：处理器、存储器、总线、I/O 设备和接口。

软件包括：嵌入式操作系统、嵌入式中间件、嵌入式应用系统等。



4. 嵌入式系统的开发流程



第二章 Linux 系统介绍

5. Linux 的特点

- ①开放性
- ②多用户
- ③多任务
- ④良好的用户界面
- ⑤设备独立性
- ⑥丰富的网络功能
- ⑦可靠的系统安全
- ⑧良好的可移植性
- ⑨丰富的应用软件支持

6. 命令行 linux 常用命令-1

- ①ifconfig:查看当前网络的设置，也可以修改当前网络的设置。

ifconfig[interface]和 ifconfig interface[aftype]option|address

其中，第一种格式为查看当前网络的设置第二种格式为修改当前网络的设置。

- ②ifup:用于重新启动指定的网络设备。

ifup 网络设备文件名

- ③ifdown:用于关闭指定的网络设备。

ifdown 网络设备文件名

- ④ethtool:查看或者修改指定网卡的工作方式。

Eg.ethtool eth0

- ⑤mii-tool:用于查看或修改指定网卡的通信协商方式。

- ⑥route:用于查看或修改主机和网络的路由信息。

- ⑦netstat:显示本机上与 IP、TCP、UDP 和 ICMP 相关的统计数据，该命令经常被用于检验本机各端口的网络连接情况。

- ⑧ping:检查网络连接的情况，ping 命令执行时使用 ICMP 传输协议，给目标主机发出

要求回应的信息，若目标主机的网络功能没有问题，就会返回回应信息。当目标主机无效或者禁用 ping 命令时，ping 命令的执行会失败。

⑨service:将目录"/ect/init.d/"中有关网络服务或系统服务脚本程序以一种统一的格式执行。

第三章 Linux 常用命令

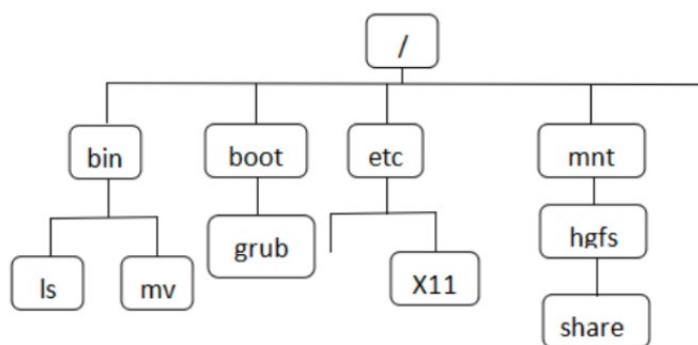
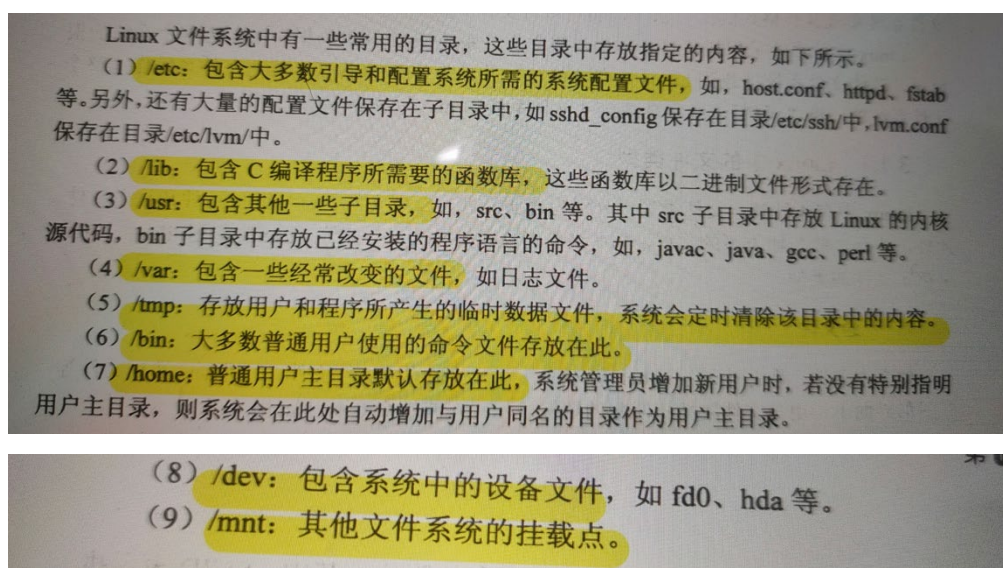
7.Linux 中的文件类型

4 种基本文件类型:普通文件、目录文件、符号链接文件和设备文件。此外，还有一些其他类型的文件（命名管道文件、socket 文件）。

❤符号:

普通文件(-)、目录文件(d)、软链接(字符链接 L)、套接字文件(S)、字符设备(c)、块设备(B)、管道文件(命名管道 P)

8.常用目录



倒树状结构

Linux 中表示文件夹: /bin/ls

Windows 中表示文件夹: D:\vm\redflag4

9.绝对路径和相对路径

①相对路径:从当前目录出发到指定目录所形成的目录名字符串。

②绝对路径:从根目录出发到指定目录所形成的目录名字符串。

10. 特殊的目录

- (1) / : 表示根目录。
- (2) . : 表示当前目录。
- (3) .. : 表示当前目录的上级目录。
- (4) ~ : 表示用户家目录。

11. 对文件操作的用户分类

所有用户 **a** 分为以下三类:

- ①文件的所有者 **u**
- ②同组用户 **g**
- ③其他用户 **o**

12. 文件操作权限

文件有以下 3 种基本操作权限:

- ①读操作 **r**
- ②写操作 **w**
- ③执行操作 **x**

13. 文件的授权属性

有两种表示形式:

- ①字母表示

Eg. **rwxr-xr--**

- ②8 进制数表示

Eg. **754** // 转换为二进制数为 **111101100**, 表示

14. 扩展操作权限 (SUID,SGID)

两种表示形式:

- ①字母表示:

< Linux 系统在基本操作权限的基础上扩展了 **SUID**、**SGID** 和粘附位, 其中, **SUID** 表示执行文件时, 将执行该文件的用户 **UID** 作为文件的 **UID**; **SGID** 表示执行文件时, 将执行该文件的用户 **GID** 作为文件的 **GID**; 粘附位表示只有文件主和 **root** 用户才能删除带有该标识的文件, 其他用户虽然有“**w**”权, 但仍然不能删除带有该标识的文件。

SUID 和 **SGID** 用字母 **s** 表示, 分别与文件主和同组用户的“**x**”位组合, 表示文件是否具有 **SUID** 或 **SGID** 位。组合时, 若该位无“**x**”权限有“**s**”权限, 则用大写字母“**S**”表示; 若该位既有“**x**”权限又有“**s**”权限, 则用小写字母“**s**”表示。粘附位用字母“**t**”表示, 与其他用户权限的“**x**”位组合, 若该位无“**x**”权限有“**t**”权限, 则用大写字母“**T**”表示; 若该位既有“**x**”权限又有“**t**”权限, 则用小写字母“**t**”表示。

例如, **rwsr-Sr-T** /// 文件主具有读、写和执行权限, 同时具有 **SUID** 位; 同组用户仅具有读权限, 同时具有 **SGID** 位; 其他用户仅有读权限, 同时具有粘附位。

- ②数字表示:

3 个扩展权限自成一组与基本权限一起构成每组 3 位共 4 组二进制数字来表示文件的授

权属性。用数字表示时，扩展权限组在其他组前面，组内的顺序依次为 SUID、SGID 和粘附位。

SUID SGID 粘附位

例如，5754 //第 1 组二进制数为 101，表示有 SUID 位，无 SGID 位，有粘附位；第 2 组二进制数为 111，表示文件主对文件具有 rwx 权限；第 3 组二进制数为 101，表示同组用户对文件有 r 权，无 w 权，有 x 权；第 4 组二进制数为 100，表示其他用户对文件具有 r 权，无 w 权，无 x 权。

15. 文件名通配符

- ①"*":任意个字符。
- ②"?":1 个字符。
- ③"[]":所包括的字符。

16. linux 常用命令-2

(1) 文件类

- ①pwd:显示当前的工作目录。
- ②cd:进入指定目录。cd [路径]
- ③ls:列出指定目录或者当前目录下的文件名。ls [选项] [文件名列表]

♥a.选项

-l: 每行显示一个文件的详细信息，称为以长格式显示，该选项最常用。若不加该参数，ls 将在一行中显示多个文件名，并以不同颜色来标记不同类型的文件。

该参数所显示的文件信息依次是文件类型、授权属性、链接数、文件属主、文件属组、文件大小、文件建立或最后修改时间、文件名。

对于符号链接文件，显示的文件名之后有“->”和指向的文件名；对于设备文件，其“文件大小”字段显示主、次设备号，而不是文件大小。

用 ls -l 命令显示的文件信息中，开头是由 10 个字符构成的字符串，其中第 1 个字符表示文件类型，类型如下。

- : 普通文件。
- d: 目录文件。
- l: 符号链接文件。
- b: 块设备文件。
- c: 字符设备文件。
- p: 命名管道文件。
- s: socket 文件。

后面的 9 个字符表示文件的授权属性，分为 3 组，每组 3 位。

对于文件，链接数表示有几个文件以硬链接形式链接到同一个 iNode 上；对于目录，链

接数表示该目录中包含几个目录，因为一个目录中至少包含“.”和“..”目录，因此，目录的链接数至少为2。

- a: 显示指定目录所包含的所有文件名与目录名，包括隐藏文件与目录。
- A: 与参数-a功能相同但不列出“.”和“..”两个特殊目录。
- b: 对文件名中的不可显示字符用八进制逃逸字符显示。
- C: 一行显示多个文件名，与选项-l相冲突。
- d: 如果后面参数是目录文件名，只显示其名称而不显示其下的各文件，经常与-l选项一起使用，以得到目录的详细信息。
- t: 显示时按修改时间（最近优先）排序。若文件修改时间相同，则按文件名升序排序。修改时间取决于是否使用了c或u选项。缺省的时间标记是最后一次修改时间。
- u: 显示时按文件上次存取的时间（最近优先）排序。即将-t的时间标记修改为最后一次访问的时间。
- s: 给出每个目录项所用的块数，包括间接块。
- f: 不排序。该选项将使-ls选项失效，并使au选项有效。
- F: 在目录名后面标记“/”，可执行文件后面标记“*”，符号链接文件后面标记“@”，命名管道文件后面标记“|”，socket文件后面标记“=”。
- i: 在输出的第一列显示文件的iNode号。
- L: 若指定的名称为一个符号链接文件，则显示链接所指向的文件。
- m: 输出按字符流格式，文件跨页显示，以逗号分开。
- n: 输出格式与-l选项相同，只不过在输出中文件属主和属组是用相应的UID和GID号来表示，而不是用字符串表示的名称。
- o: 与选项-l相同，只是不显示文件主的名称。
- p: 在目录后面加一个“/”。
- q: 将文件名中的不可显示字符用“?”代替。
- r: 按字符ASCII值降序显示文件名。
- R: 递归式地显示指定目录的各个子目录中的文件名。
- x: 以按行排序形式显示文件名信息。

(2) 使用举例。

```
ls //以每行多个文件名的方式，列出当前目录所包含的文件名。
ls -F /home/bright /var //分别列出目录/home/bright和/var所包含的文件名，并在文件名后用指定字符指出文件的类型，例如*代表可执行文件。
ls -aF /home/bright //以带标记的形式列出目录/home/bright所包含的全部文件名，包括隐藏文件。
ls -laF /home/bright //以带标记的长格式列出目录/home/bright所包含的全部文件名，包括隐藏文件。
ls -laFin /home/bright //在上面命令的基础上，首先显示iNode号，并且把文件属主和属组以UID号和GID号的形式显示。
```

④cp:将给出的文件或目录拷贝到指定的文件或目录中。cp[选项] 源文件 目标文件

♥ 选项:

- a: 该选项通常在拷贝目录时使用，它保留链接、文件属性，并递归地拷贝子目录中的内容，其作用等于dpr选项的组合。
- d: 拷贝时保留链接。
- p: 除复制源文件的内容外，还将把其最后修改时间和访问权限也复制到目标文件中。
- r: 若源文件是目录文件，cp将递归复制该目录下所有的子目录和文件，目标文件名必须为一个目录文件名。
- l: 不作拷贝，只是链接文件。

(2) 使用举例。

cp 1.txt /home/bright/2.txt //将当前目录中的文件1.txt复制到目录/home/bright/中，文件名为2.txt。

cp -r /home/bright/cml /home/bright/yxj //将/home/bright/cml目录中的所有文件及其子目录拷贝到目录/home/bright/yxj中。

cp /home/user/*.txt //将/home/user/目录下以.txt为后缀的文件复制到当前目录中。

⑤mv:将指定文件或目录改名或将指定文件或目录进行移动。mv[选项] 源文件 目标

文件

(1) 选项。

-i: 交互方式操作, 如果 mv 操作将导致对已存在目标文件的覆盖, 则系统要求用户回答“y”进行文件覆盖或“n”放弃覆盖。

-f: 禁止交互操作, 在 mv 操作要覆盖已有的目标文件时不给任何提示而直接覆盖目标文件。

-u: 只有在源文件比目标文件新, 或者目标文件不存在时, 才执行移动功能。

(2) 使用举例。

```
mv /home/user/data/ /var/tmp
//将/home/user/data/目录移动到/var/中, 目录名变更为 tmp。
mv /home/user/1.txt /var/tmp/1.txt
//将文件/home/user/1.txt 移动到目录/var/tmp/中, 文件名依然为 1.txt。
mv /home/user/2.txt 20.txt
//将文件/home/user/2.txt 重命名为 20.txt, 文件的位置不变。
```

6. rm

Linux 系统中随时会有文件或者目录变得过时且毫无用处,此时可以用 **rm** 命令将过时文件或者目录删除。该命令可以一次删除一个或多个文件或目录,对于链接文件,只是删除链接,原有文件保持不变。

rm 命令的格式为:

rm [选项] 文件

其中,选项是对 **rm** 命令要执行功能的进一步说明,文件表示要删除的文件名。

(1) 选项。

-f: 删除过程中直接删除指定的文件或子目录而不需要进行确认。

-r: 将指定的目录及其子目录递归地删除,删除时需要用户确认。

-i: 删除文件或者目录前进行确认,输入 **y** 或者 **yes** 进行删除,输入 **n** 或者 **no** 或者其他字符则放弃删除。

(2) 使用举例。

```
rm /home/user/20.txt //删除目录/home/user/下的文件 20.txt
rm -r /var/tmp/ //以递归形式删除目录/var/tmp/及其包含的文件和子目录。
rm -i a* //以交互确认的形式删除当前目录下以字符 a 开头的文件。
rm -r -f /home/user/tmp/ //以递归方式删除目录/home/user/tmp/及其所包含的文件和子目录,
删除过程中不需要确认,直接删除。
```

7. cat

显示文本文件的内容。该命令后可以跟多个文本文件名,将依次显示每个文件的内容。

• **cat** 命令的格式为:

cat [选项] 文件

其中,选项是对 **cat** 命令的执行功能的进一步说明,文件表示要显示内容的文件名。

(1) 选项。

-v: 用特殊字符形式显示控制字符,但 **LFD** 与 **TAB** 除外,加了 **-v** 参数后, **-T** 和 **-E** 参数将起作用。

-T: 将 **TAB** 显示为 “**ùI**”,该参数需要与 **-v** 一起使用,如果没有使用 **-v**,则 **-T** 将被忽略。

-E: 每行的末尾将显示一个 **\$** 符,需要与 **-v** 一起使用。

-u: 输出不经过缓冲区。

-A: 等于 **-vET**。

-t: 等于 **-vT**。

-e: 等于 **-vE**。

(2) 使用举例。

```
cat 1.txt //在屏幕上显示 1.txt 文件的内容。
cat -A 1.txt //显示文件 1.txt 的内容,同时显示文件中所包含的特殊字符。
cat 1.txt 2.txt //依次显示 1.txt 和 2.txt 文件的内容。
cat test[123] //依次显示文件 test1、test2 和 test3 的内容。
```

8. head

head 命令用于查看指定文本文件开头的内容，head 命令的格式为：

head [选项] 文件

其中，选项是对 head 命令要执行功能的进一步说明，文件表示要查看内容的文件列表。

(1) 选项。

-c: 指明要查看文件的前多少个字符。

-n: 指明要查看文件的前多少行。

-q: 在文件内容前不显示文件名标识。

-v: 在文件内容前显示文件名标识。

(2) 使用举例。

```
head -c 100 1.txt // 显示当前目录下文件 1.txt 的前 100 个字符。
```

```
head -n 5 -v 1.txt 2.txt 3.txt //显示当前目录下文件 1.txt、2.txt、3.txt 的前 5 行内容，  
在每个文件的内容前显示相应的文件名标志。
```

9. tail

tail 命令用于查看指定文本文件尾部的内容，tail 命令的格式为：

tail [选项] 文件

其中，选项是对 tail 命令要执行功能的进一步说明，文件表示要查看内容的文件列表。

(1) 选项。

-c: 指明要查看文件的后多少个字符。

-n: 指明要查看文件的后多少行。

-q: 在文件内容前不显示文件名标识。

-v: 在文件内容前显示文件名标识。

+: 从指定行以后开始显示。

-: 从距文件尾部的指定行处开始显示。

(2) 使用举例。

```
tail -c 100 1.txt // 显示当前目录下文件 1.txt 的后 100 个字符。
```

```
tail -n -v 5 1.txt 2.txt 3.txt //显示当前目录下文件 1.txt、2.txt、3.txt 的后 5 行内容，  
在每个文件的内容前显示相应的文件名标志。
```

```
tail +3 1.txt // 从第 3 行开始显示当前目录下文件 1.txt 的内容。
```

```
tail -3 1.txt // 显示当前目录下文件 1.txt 的后 3 行内容。
```

10. more

more 命令用于分屏显示文本文件的内容，当一满屏文本内容显示完之后，暂停显示并在屏幕底部打印 "-- More--" 和已显示文本占全部文本的百分比标记，若要继续显示，按回车键屏幕内容向上滚动一行或按空格键屏幕内容向上滚动一屏。

more 命令的格式为：

more [选项] 文件

真要记这么多吗？

17. 输入输出和转向

3.3.1 输入转向

在执行命令或者运行程序时，经常需要从键盘进行大量的输入。如果输入过程中出现错误，则前面输入的内容可能要废弃，重新进行输入。用户可以将命令或者程序运行时所需要输入的数据事先存放在一个文本文件中，在命令或者程序运行时，利用输入转向的功能让运行中的命令或者程序从指定的文本文件中读取，减少数据输入过程中的错误。输入转向的符号为“<”。

例如，用户自编的程序 `score` 在运行时需要输入大量数据，用户可以事先将程序运行时所需要的数据存放在文本文件 `mydata` 中，程序 `score` 运行的命令格式可以为：

```
./score <mydata
```

则 `score` 运行中所需要的数据从文件 `mydata` 中进行读取。

3.3.2 输出转向

Linux 中命令或者程序运行时会在屏幕上显示相应的信息，利用输出转向可以将命令或者程序运行中所产生的信息保存在文件中，输出转向符为“>”，当输出转向符写为“>>”时，表示将输出内容追加到指定文件的后面。输出转向符的使用如下所示。

```
ls -la >ls.output //将命令 ls -la 的输出转向存储到文件 ls.output 中，若文件 ls.output 不存在，则创建；若文件 ls.output 已经存在，则覆盖。  
cat 1.txt >>ls.output //将命令 cat 1.txt 的输出以追加的方式转向到文件 ls.output 中，若文件 ls.output 不存在，则创建；若文件 ls.output 已经存在，则将该命令的输出追加到文件 ls.output 的末尾。  
./score <mydata >myoutput //程序 score 运行时所需输入数据从文件 mydata 中获取，输出信息转向存储到文件 myoutput 中。
```

3.3.3 管道

管道是进程间通信的一种手段，分为无名管道和命名管道。无名管道仅存在于内存中，不产生额外文件，而命名管道会产生相应的管道文件。无名管道可以在命令行中实现而命名管道必须通过编程实现。

无名管道能够实现在一个命令行中，将前一个命令的输出作为后一个命令的输入，不需要保存命令执行的中间结果，实现在一个命令行中同时执行两个命令。管道符号为“|”，举例如下：

```
ls -la |grep test.zip // 将命令 ls -la 的输出作为命令 grep 的输入，表示在命令 ls -la 的执行结果中查找是否有字符串 test.zip，也就是在当前目录下查找文件 test.zip，若存在，则显示文件名，若不存在，屏幕无显示。  
cat test.c | more // 以分屏的形式显示文本文件 test.c 的内容。
```

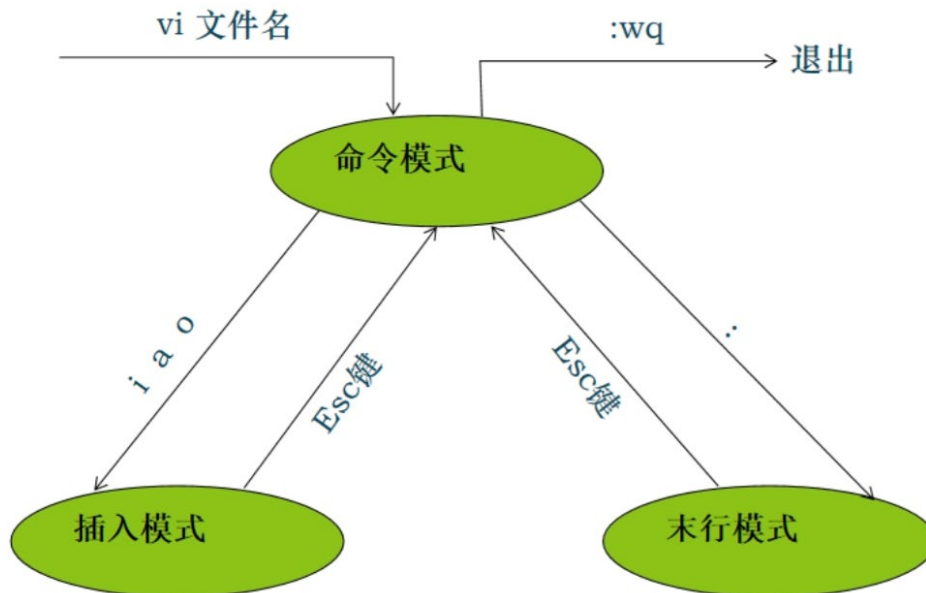
18.vi 编辑模式

三种工作模式：vi+文件名可以进入 vim 模式，:wq 可以退出 vim

①一般模式：进入 vim 就是一般模式，只能通过按键向编辑器发送命令，不能输入文字。

②编辑模式（插入模式）：在一般模式下按 i (I) 或 a 就会进入，此时可以输入文字，输入结束可以按 Esc 返回到一般模式

③命令模式：在一般模式下，输入“:”即可进入命令模式，左下角会有一个冒号出现，此时可输入命令并执行，按 **Esc** 返回到一般模式。



19. 与嵌入式开发相关的 Linux 网络服务

①TFTP（简单文件传输协议）是 TCP/IP 协议族中的一个用来在客户机与服务器之间进行简单文件传输的协议，它基于 UDP（User Data Protocol）协议实现，使用超时重传方式来保证数据的到达。

②Telnet 是一个基于字符界面的远程登录协议。通过 Telnet，用户可以远程登录到指定的主机中，以本地主机为仿真终端对远程主机进行操作。

③NFS（网络文件系统）是一种将远程主机上的指定目录经网络挂载到本地系统的一种机制，通过网络文件系统，用户可以在本地系统上像操作本地目录一样来对远

程主机的目录进行操作。

要实现 NFS 功能，首先需要在服务器端进行配置，然后启动相应的服务后，客户端就可以挂载服务器端所指定的目录到本机的挂载点目录。

RPC（远程过程调用协议），NFS 可以看成是 RPC 服务的一个应用。NFS 是个文件系统，而 RPC 负责信息的传输。

rpc.nfsd，即 NFS 服务本身。作用是管理客户端能否登录服务器。

rpc.mountd，即 mountd 服务。作用是管理 NFS 的文件系统。

rpc.rquotad，即 rquotad 服务。作用是对远程用户提供用户配额信息。

步骤如下：

- ①查询 NFS 服务的软件包是否安装：
- ②开启 NFS 服务
- ③配置 NFS 服务
- ④启动 NFS 服务
- ⑤定制防火墙

步骤 1：在/etc/services 文件中指定 mountd 和 rquotad 服务的端口为 1011 和 1012。

步骤 2：存盘退出后，要重启服务。

步骤 3：定制防火墙

步骤 4：重启防火墙

⑥验证 NFS 服务

⑦使用 NFS 服务

第五章 Linux 中的 C 语言编程

20.工具

GCC -- GNU C 编译器

GDB -- GNU 调试工具

Make -- 程序维护工具

21.GCC 概念

①是由 GNU 开发的编程语言编译器。

②GNU 是一个开源社区，它的目标是创建一套完全自由的操作系统。

③是一个强大的工具集合，包含预处理器-cpp、编译器-ccl、汇编器-as、链接器-ld 等组件。

22.GCC 所支持的文件名后缀

文件名后缀	文件类型
.c	c语言源程序文件
.C .cpp .cc .c++ .cxx	c++源文件
.h	头文件
.i	预处理后的c源文件
.s	汇编程序文件
.o	编译后的目标文件
.a	静态链接库
.so	动态链接库

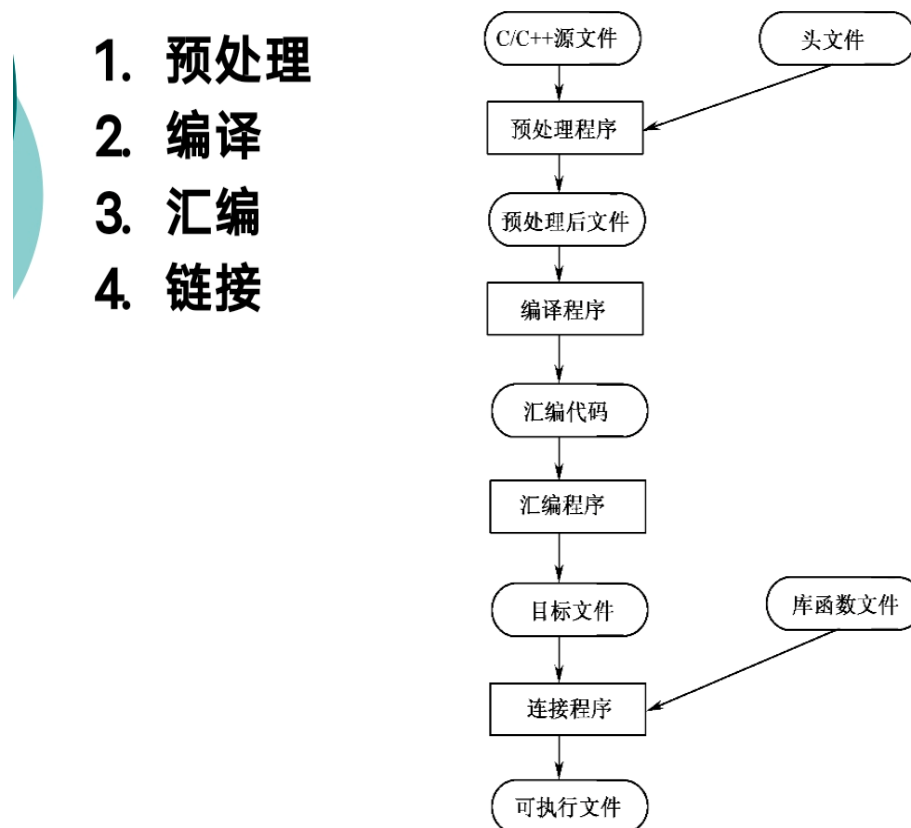
.ii:预处理后的 c++源文件

.m:Object C 源程序文件

.s:汇编语言源程序文件

.S:已经预处理过的汇编语言源程序文件

23.GCC 编译过程



由上图可知：

①预处理（**Preprocessing**）：对源程序中的预处理命令和特殊符号进行处理。

头文件包含指令（**# include ...**）

宏定义指令（**# define ...**）

条件编译指令（**# ifdef ... # endif**）

②编译（预处理文件到汇编代码）:将预处理后的 C 语言程序转换为汇编语言程序

③汇编（汇编代码到目标代码）:将汇编语言程序转换为机器语言。

④链接（目标代码到可执行代码）:将多个机器语言程序链接为可执行程序

24.GCC 的使用格式: `gcc [options] filenames`

①-E 仅作预处理，处理结果在标准输出设备（显示器）输出，生成.i 文件。

②-S 编译到汇编语言，生成.s 文件。

③-c 编译生成以.o 为后缀的目标文件，不生成可执行文件。

④-o 该参数后跟要生成的可执行文件的名称，该参数缺省时生成的可执行文件名称为 `a.out`。

⑤-Wall 编译时显示警告信息

⑥-g 生成符号调试工具 GDB 所需的符号信息，要想用 GDB，必须加入这个参数。

⑦ -lname: 链接函数库文件“libname.a”。

⑧-Ldir 将 dir 所指出的目录加到“函数库搜索列表”中

【例子】

`#gcc -E hello.c` (在屏幕输出)

`#gcc -S hello.c` (自动生成 hello.s)

`#gcc -c hello.c` (自动生成 `hello.o`)

`#gcc hello.c -o hello` (不加 `-o` 参数, 生成文件 `a.out`)

`#gcc -Wall hello.c` (输出警告信息)

【例子 2】

例 2: 编译一个求正弦函数值的 C 语言程序

(`test2lib.c`)。

提示: 正弦函数用到的库文件是 `libm.a`.

该库文件存放位置是: `/usr/lib/libm.a`

编译命令如下:

`# gcc -Wall test2lib.c /usr/lib/libm.a`

`# gcc -Wall test2lib.c -lm`

【例子 3】

例 3: 编译一个含两个 C 语言程序的小项目 (`test3main.c`
`test3foo.c`)。

编译命令如下:

`#gcc test3main.c test3foo.c`

25.GDB (GNU Debugger) 的概念

- ①GNU 发布的 Linux 下的程序调试工具
- ②可以使程序开发者在程序运行时观察程序的内部结构
- ③是一种命令行调试工具

26.GDB 所提供的功能

- ①设置断点

- ②监视、修改变量
- ③单步执行
- ④显示/修改寄存器的值
- ⑤查看堆栈
- ⑥远程调试

27.利用 GDB 调试的前提

在用 GCC 编译程序过程中，使用-g 选项，生成 gdb 工作所必须的调试信息；

```
#gcc -g test.c -o test
```

28.启动 gdb 的方法 :gdb filename

说明：filename 是可执行文件，而不是源代码文件。

29.GDB 的基本命令

①显示源程序

list 命令：列出调试程序的源代码，简写为 l。

格式	说 明
list 或 l	以每页10行的形式分页显示源程序
list start,end	显示从start到end行的代码

【例子】

```
list -          //显示当前行之前的 10 行
list +          //显示当前行之后的 10 行
list 2,10       //显示 2 到 10 行
show listsize   //显示行数设置
```

set listsize 5 //修改行数设置为 5

②设置断点

break 命令：设置断点，简写为 **b**。

【例子】

break 10 //在第 10 行设置断点

b main //在函数入口处设置断点

③显示断点信息

info break 命令：显示断点信息，简写为 **i break**。

【例子】

i break //显示程序中的所有断点信息

④运行程序

run 命令：从头运行程序，执行到断点处，或者直到程序结束。简写为 **r**。

next 命令：单步执行，不进入函数内部。简写为 **n**。

step 命令：单步执行，进入函数内部。简写为 **s**。

continue 命令：程序从当前位置开始，执行到断点处，或者直到程序结束。简写为 **c**。

⑤查看运行时数据

print 命令：显示表达式的值。简写为 **p**。

whatis 命令：显示变量的数据类型。

ptype 命令：显示指定变量的数据类型。比 **whatis** 功能

更强，可以显示结构体的定义。

⑥删除断点

clear 命令：删除指定断点。后跟断点所在行号或函数名。

delete 命令：删除指定断点。后跟断点编号，不指定编号时，删除所有断点。简写为 **d**。

disable 命令：使指定断点失效，后跟断点编号。

enable 命令：使指定断点生效，后跟断点编号。

⑦其他命令

回车键：继续执行上一条命令。

quit 命令：退出 **gdb**，简写为 **q**。

set 命令：修改变量的值。

【例子】

```
set variable i=9
```

```
print i=9
```

30.维护工具 make 的概念

①是 **GNU** 推出的用来管理多个程序的工具

②依靠 **Makefile** 文件中定义的规则来实现管理

③适合于支持多文件构成的大中型软件项目的编译、链接、清除中间文件等管理工作

③提供和识别多种默认规则，方便对大型软件项目的管理

④make 可以实现“自动化编译”。

⑤make 本身是一个命令，该命令的执行需要一个 Makefile 文件。

⑥Makefile 文件的命名有三种：GNUmakefile、makefile、Makefile，执行顺序从左至右。建议用后两种的其中一种即可。

31.make 工作机制

【例子】

题目：



假如有三个源文件program.c、pro1.c、pro2.c，program.c文件包含有自定义的头文件lib.h，要求生成可执行文件program。

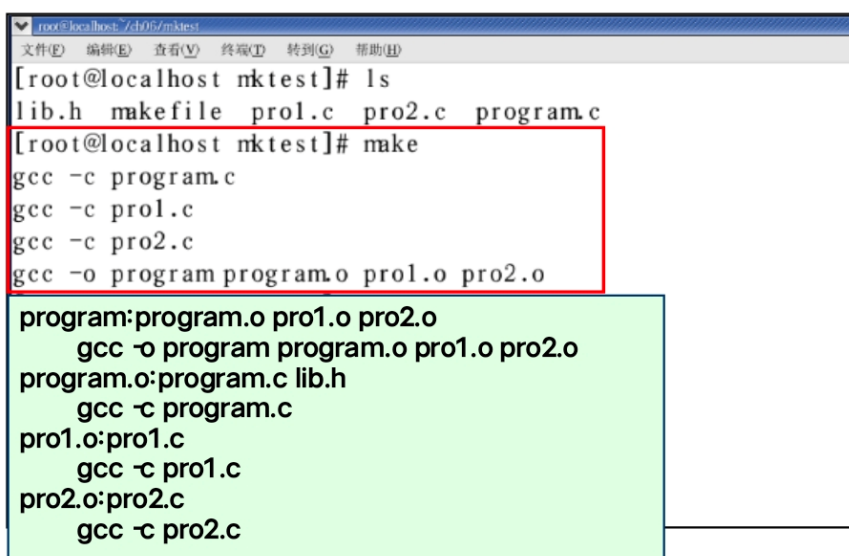
```
//lib.h
void pro1(char *);
void pro2(char *);
```

```
//program.c
#include "lib.h"
int main()
{
    pro1("this is pro1.");
    pro2("this is pro2.");
    return 0;
}
```

解答：

编译方式3: 编写makefile文件

```
program:program.o pro1.o pro2.o
    gcc -o program program.o pro1.o pro2.o
program.o:program.c lib.h
    gcc -c program.c
pro1.o:pro1.c
    gcc -c pro1.c
pro2.o:pro2.c
    gcc -c pro2.c
```



```
root@localhost ~/ch06/mktest
文件(F) 编辑(E) 查看(V) 终端(T) 转到(G) 帮助(H)
[root@localhost mktest]# ls
lib.h  makefile  pro1.c  pro2.c  program.c
[root@localhost mktest]# make
gcc -c program.c
gcc -c pro1.c
gcc -c pro2.c
gcc -o program program.o pro1.o pro2.o

program:program.o pro1.o pro2.o
    gcc -o program program.o pro1.o pro2.o
program.o:program.c lib.h
    gcc -c program.c
pro1.o:pro1.c
    gcc -c pro1.c
pro2.o:pro2.c
    gcc -c pro2.c
```

32. Makefile 的书写规则

① Makefile 的规则:

定义要创建的目标文件

指出要生成目标文件文件的依赖文件

依赖文件创建目标文件的命令

② 规则的格式

必须是制表符，
不能是空格

目标文件： 依赖文件列表
<tab>命令

③说明：

- a.在命令行输入命令： `make`，可直接生成可执行文件 `program`。`program` 是默认目标。
- b.`make` 命令执行的是当前目录下的 `Makefile`。
- c.`make` 只负责维护文件的依赖性。它会比较目标文件和依赖文件的修改日期，从而判定是否执行该规则所定义
的命令。

若目标文件不存在，则执行；

若目标文件已存在，但是依赖文件比目标文件新，则
执行；

否则不执行。

【例子】步骤

```
hello:main.o foo.o
    gcc main.o foo.o -o hello
main.o:main.c
    gcc -c main.c
foo.o:foo.c
    gcc -c foo.c
clean:
    rm hello main.o foo.o
```

- 1、输入 make 命令后，make 会在当前目录找名字为“Makefile”或者“makefile”的文件。
- 2、如果找到后，它会找文件中的第一个目标文件（hello），把这个文件作为最终的目标文件。（即第一个目标为 make 的默认目标）。

◆执行 make ， 与 make hello 功能相同.

- 3、如果 hello 不存在，那么就执行后面所定义的命令来生成 hello 这个文件。
- 4、如果 hello 存在，但是依赖中的.o 文件修改时间比 hello 这个文件新，那么也会执行后面所定义的命令来更新 hello 文件。
- 5、如果 hello 所依赖的.o 文件不存在，那么 make 会在 Makefile 文件中找到该.o 文件的依赖性，如果找到，再根据这个依赖规则生成这个.o 文件。

6、以此类推，一层一层的找文件的依赖关系，直到最终编译出一个 `hello` 文件。

7、如果在找寻的过程中，出现了错误（如依赖的文件找不到），那么 `make` 就会直接退出，并报错。但是如果是规则中的命令的错误，或者编译不成功，`make` 是根本不理的。`make` 只负责管理文件的依赖性。

总结：由此可以看出，像 `clean`，它没有被第一个目标文件直接或间接关联，那么 `clean` 后面定义的命令就不会被自动的执行。如果需要执行，就要明确指明它。

❤ `clean` 这条规则的作用是：清空目标文件（可执行文件 `hello` 和 `.o` 这些中间文件）

目的是：便于重编译，也利于保持文件的清洁。

一般的风格是：

`clean:`

`rm hello *.o`

`clean` 的规则不要放在 `Makefile` 文件的开头。

33. 伪目标的概念

`clean` 就是一个伪目标。

伪目标并不是一个文件，而只是一个标签。

伪目标的执行不会生成相应的目标文件。

伪目标放在后面，不会自动执行（如需执行需要明确指出）。

伪目标也可作为默认目标（把它放在第一个规则中即可）。

34.常用的伪目标 all

```
all:  
    gcc fun1.c fun2.c main.c -o  
    test  
clean:  
    rm -f test
```

◆当执行 `make` 时，从当前目录读入 `Makefile` 文件，开始处理第一个规则，该规则的目标称为缺省目标。上述 `Makefile` 中的缺省目标是 `all`。

◆在执行 `make` 的时候，可以指定目标执行，如 `make clean`。即 `make` 会直接读入执行目标 `clean`，跳过 `all`。

◆执行 `make all`，与 `make` 功能相同。

常用的伪目标all

```
all:prog1 prog2 prog3

prog1:prog1.o utils.o
    gcc prog1.o utils.o -o prog1
prog2:prog2.o
    gcc prog2.o -o prog2
prog3:prog3.o sort.o utils.o
    gcc prog3.o sort.o utils.o -o prog3
.....
```

◆**作用**是：一次生成多个可执行文件。

35.Makefile 中使用变量

①变量功能

简化 Makefile，降低错误风险。

贮存文件名列表

贮存可执行文件名

贮存编译器标志参数

②变量的命名规则

包含数字、字符、下划线（可以是数字开头）

不能含有“： # =”和空字符（空格、回车等）

区分大小写

③变量的定义格式：

变量名 = 字符串

④变量的引用:

`$(变量名)` 或者 `${变量名}`

36. 预定义变量

预定义变量

预定义变量	说明
<code>CC</code>	c编译器的名称，默认为cc
<code>CFLAGS</code>	c编译器的选项，无默认值
<code>CPP</code>	c程序的预处理器，默认为 <code>\$(CC) -E</code>
<code>RM</code>	删除文件命令，默认是rm -f
<code>EXEC</code>	可执行程序名称
<code>SRCS</code>	源代码
<code>OBJS</code>	目标文件

例子

```
object = program.o pro1.o pro2.o
CC = gcc
CFLAGS = -g
program: $(object)
    $(CC) -o program $(object) $(CFLAGS)
```

37. 内部变量

内部变量（自动化变量）

变量名	含义
<code>\$@</code>	当前目标文件的名称
<code>\$?</code>	比当前目标文件新的依赖文件列表 (当前目标文件所依赖的文件)
<code>\$<</code>	规则中第一个依赖文件
<code>\$\$</code>	所有依赖文件

例子:

```
object= program.o pro1.o pro2.o
program: $(object)
    gcc $$ -o $@
```

38. 隐式规则

不需要在 makefile 中写出来的规则。

可以简化 makefile 文件的内容。

♥几个常用的隐式规则

C 语言程序：.o 文件会自动找到相应的.c 文件，用 cc 命令进行编译

C++程序：.o 文件会自动找到对应的.cc 文件，用 g++进行编译

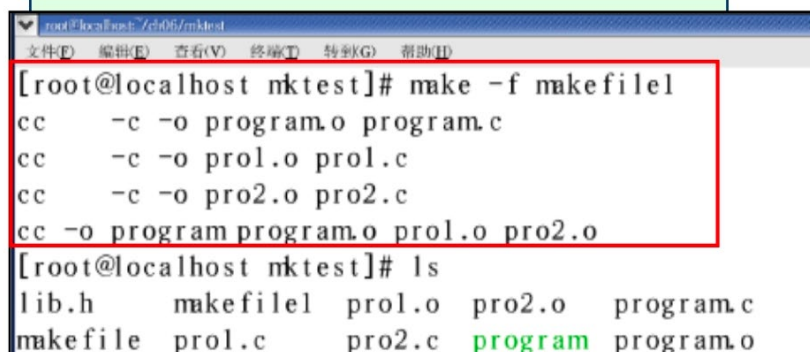
汇编程序：.o 文件会自动找到对应的.s 文件，并且用 as 命令进行汇编。

说明:有了这些隐式规则，用户只需在 makefile 中指定依赖关系即可。

【例子】

例子：简化后的makefile

```
object=program.o pro1.o pro2.o
program: $(object)
    $(CC) -o program $(object)
program.o:program.c lib.h
pro1.o:pro1.c
pro2.o:pro2.c
clean:
    rm *.o
```



```
root@localhost: ~/mktest
文件(F) 编辑(E) 查看(V) 终端(T) 转到(G) 帮助(H)
[root@localhost mktest]# make -f makefile1
cc      -c -o program.o program.c
cc      -c -o pro1.o pro1.c
cc      -c -o pro2.o pro2.c
cc -o program program.o pro1.o pro2.o
[root@localhost mktest]# ls
lib.h      makefile1  pro1.o    pro2.o    program.c
makefile   pro1.c     pro2.c    program   program.o
```

例子：进一步简化makefile

```
object=program.o pro1.o pro2.o
program: $(object)
    $(CC) -o program $(object)
clean:
    rm *.o
```

39.追加变量的值

方法：使用“+=”操作符给变量追加值。

【例子】

```
objects=main.o foo.o bar.o utils.o
```

```
objects += anther.o
```

于是，\$(objects)就为 main.o foo.o bar.o utils.o anther.o

注意：如果变量之前没有定义过，那么“+=”会自动变成“=”，给前面的变量赋值；如果变量之前有定义，那么“+=”就是追加赋值。

40.Makefile 中有什么

①显式规则：该规则明确的写出了要生成的文件（目标），依赖的文件（依赖）和生成所用的命令。

②隐式规则：使用的是 make 的自动推导功能。请参阅《跟我一起写 Makefile》文档。

③变量的定义：变量一般都是字符串，当 Makefile 被执行时，这些变量会被扩展到相应的引用位置上，类似于 C 语言中的宏。

④注释：跟 Linux 的 shell 脚本一样，注释使用#字符。

【重点例子】

题目:一个程序包括以下内容。

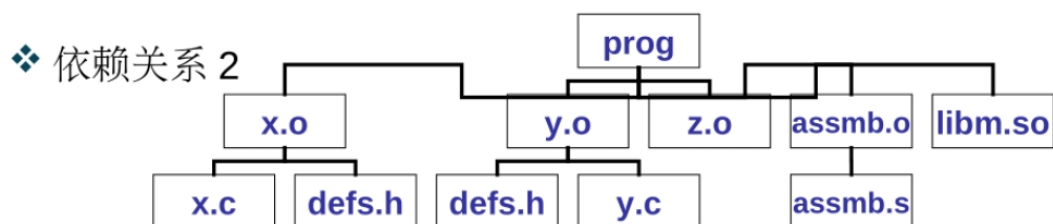
■ 三个 C 语言源文件：x.c y.c z.c。

·一个头文件 defs.h:x.c 和 y.c 都使用了 defs.h 中的声明。

■ 汇编语言文件：assmb.s

■ 一数学运算函数库：/home/mqc/lib/libm.so

解答:



❖ makefile 文件:

```
prog:x.o y.o z.o assmb.o
gcc x.o y.o z.o assmb.o -L /home/mqc/libm -lm -o prog
x.o: x.c defs.h
gcc -c x.c
y.o: y.c defs.h
gcc -c y.c
z.o: z.c
gcc -c z.c
assmb.o: assmb.s
as -o assmb.o assmb.s
```

第六章 ARM 体系结构和指令系统

41.ARM 处理器系列概述

经过多年的发展，ARM 处理器已经形成一个庞大的家族。ARM 公司目前支持主要系列的处理器产品如下：

- ①ARM7 系列处理器；
- ②ARM9 系列处理器；
- ③ARM9E 系列处理器；
- ④ARM10E 系列处理器；
- ⑤ARM11 系列处理器；
- ⑥Cortex 系列处理器；
- ⑦SecurCore 系列处理器。

42.哈佛和冯诺依曼的区别

- ①冯诺依曼结构:是一种将系统指令存储器和数据存储器合并在一起的存储器结构。
- ②哈佛结构:是一种将程序指令存储和数据存储分开的存储器结构
- ③区别:

a.使用两个独立的存储器模块分别存储指令和数据，每个存储模块都不允许指令和数据并存。

b.使用独立的两条总线分别作为 CPU 与每个存储器之间的专用通信路径，两条总线毫无关联

43.CORTEX 系列

- ①CORTEX-A 复杂的操作系统和大型的应用场合，
如： 高端手机，金融事务处理机等
- ②CORTEX-R 实时应用场合，
如： 大型发电机控制器，机械手臂等

③CORTEX-M 传统的低成本低功耗及速中断反应及高效率的自动控制场合，

如：医用器械，电子玩具，无线网络等

44.存储器的存储格式（大端是顺序，小端是逆序）

A. 小端存储法(0x01234567)

地址	0x8000	0x8001	0x8002	0x8003
数据（16进制表示）	0x67	0x45	0x23	0x01
数据（二进制表示）	01100111 1	01000101	00100011	00000001

B. 大端存储法

地址	0x8000	0x8001	0x8002	0x8003
数据（16进制表示）	0x01	0x23	0x45	0x67
数据（二进制表示）	00000001	00100011	01000101	01100111

45.ARM 系列处理器

嵌入式处理器的 7 种工作模式及两种工作状态

（一）7 种工作模式

①用户模式（Usr）：非特权模式，大部分任务执行在这种模式（10000）

②快速中断模式（FIQ）：当一个高优先级（fast）中断产生时将会进入这种模式（10001）

③外部中断模式（IRQ）：当一个低优先级（normal）中断产生时将会进入这种模式（10010）

④管理模式（svc）：当复位或软中断指令执行时将会进入这种模式（10011）

⑤中止模式（**Abt**）：当存取异常时将会进入这种模式
(10111)

⑥未定义模式（**Und**）：当执行未定义指令时会进入这种模式（11011）

⑦系统模式（**Sys**）：使用 and **User** 模式相同寄存器集的特权模式（11111）

♥除用户模式外其他模式都是特权模式，特权模式下程序可以访问所有系统资源，也可以随意进行处理器切换。

♥特权模式中除系统模式外其余五种又称异常模式，在每一种异常模式中都有一组寄存器，供相应的异常处理程序使用。

♥大多数程序运行于用户模式硬件权限级别：
系统模式 > 异常模式 > 用户模式

♥进入特权模式是为了处理中断、异常、或者访问被保护的系统资源。

（二） 工作状态

①**ARM 状态**：32 位，**ARM** 状态执行字对齐的 32 位 **ARM** 指令。

②**Thumb 状态**，16 位，执行半字对齐的 16 位指令。

♥**ARM** 和 **Thumb** 两种状态之间的切换不影响处理器的工作模式和寄存器的内容。

♥️ ARM 处理器在处理异常时，不管处理器处于什么状态，则都将切换到 ARM 状态。

♥️ 用 Bx、Rn 指令来进行两种状态的切换。

其中：

Bx 是跳转指令，

Rn 是寄存器（1 个字，32 位），

如果 Rn 的位为 1，则进入 Thumb 状态；

如果 Rn 的位为 0，则进入 ARM 状态。

处理器的模式与CPSR

313029282726876543210

NZC V Q

DNM (RAZ)

IFIT

M4M3M2M1M0

条件标识位

中断禁止

控制位

M4~M0	处理器模式	可访问的寄存器
10000	用户模式	PC, R14~R0, CPSR (只读)
10001	快速中断	PC, R14_fiq~R8_fiq, R7~R0,CPSR,SPSR_fiq
10010	外部中断	PC, R14_irq~R13_irq, R12~R0,CPSR,SPSR_irq
10011	特权模式	PC, R14_svc~R13_svc, R12~R0,CPSR,SPSR_svc
10111	中止模式	PC, R14_abt~R13_abt, R12~R0,CPSR,SPSR_abt
11011	未定义模式	PC, R14_und~R13_und, R12~R0,CPSR,SPSR_und
11111	系统模式	PC, R14~R0, CPSR (读写)

当异常发生时，在特权模式下，程序可以修改这些位

MSR CPSR_c, #0xd3

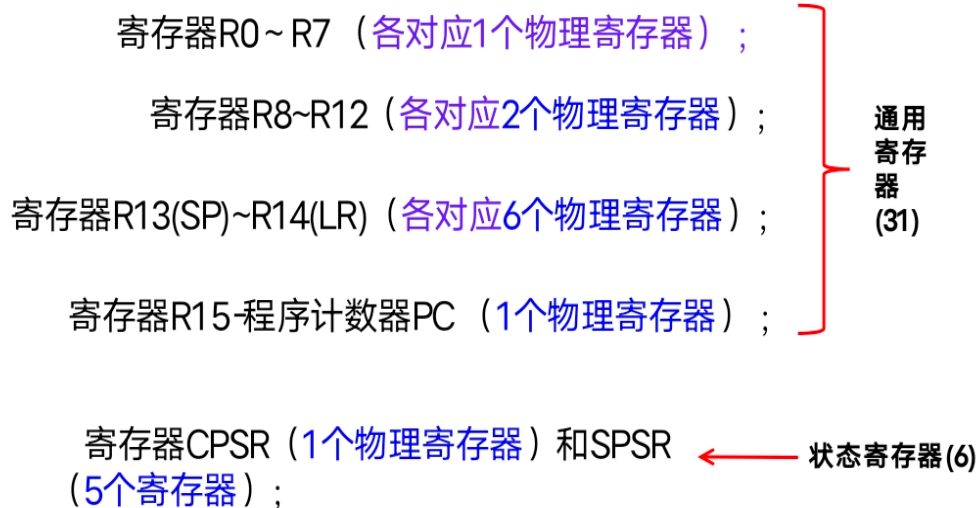
(1) 中断禁止位I、F 当I=1时禁止IRQ中断、当F=1时禁止FIQ中断

(2) T控制位：用于控制指令执行的状态，T=0表示执行ARM指令，T=1时标识执行Thumb指令；

(3) M控制位：控制处理器模式，具体含义如表所示

46. 寄存器

ARM 寄存器共有 37 个，包括 31 个通用寄存器和 6 个状态寄存器，每个寄存器都是 32 位，但是状态寄存器（CPSR）只用了其中的部分位。



①未备份寄存器

R0-R7 在所有模式下都只对应哪一个物理寄存器，未备份寄存器没有被系统用于任何特殊目的。

②备份寄存器

R8-R12，每个寄存器对应两个物理寄存器，分别用 R8-R12 和 R8_fiq-R_12fiq 表示。在快速中断模式下使用 R8_fiq-R_12fiq，在其他模式下使用 R8-R12。也未被用于特殊目的

③寄存器 R13 和 R14

也称备份寄存器，但每个寄存器对应 6 个物理寄存器。

♥♥ R13 在 ARM 中常用作堆栈指针，指令中常用 sp 表示，每个异常模式都有自己的物理 R13。应用程序在开始阶段初始化 R13，使其指向异常模式专用的对栈地址。当进入异常模式时，可以将需要使用的寄存器的值保存在 R13 所

指的堆栈中，当退出异常时，将保存的值弹出，这样使异常程序不会破坏被其中断的应用程序的现场

❤️ **R14** 被称为连接寄存器(LR)。当执行 **BL** 或 **BLX** 指令调用子程序时，各模式所对应的物理 **14** 被设置成该子程序的返回地址。因此可以通过 **MOV PC, LR** 的方式实现子程序的返回操作。

④寄存器 **R15**

用作程序计数器(PC)，用来指出当前执行指令的地址，所有模式下都只对应一个物理寄存器。由于 **ARM** 采用流水线指令，所以 **pc** 值为当前地址+8 个字节。

⑤CPSR（当前程序状态寄存器）

CPSR 在所有处理器模式下都是只有一个物理寄存器，除用户模式只能进行读操作意外，其余模式都可进行读写操作。

1)格式:

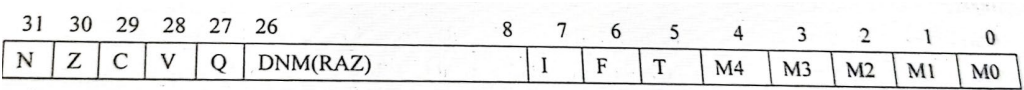


图 6-1 CPSR 寄存器格式

2)各标志位的含义:

❤️ **N**: 如果结果是带符号二进制补码，那么，若结果为负数，则 **N=1**;若结果为正数或 0，则 **N=0**。

❤️ **Z**: 若指令的结果为 0，则置 1(通常表示比较的结果为“相等”)，否则置 0。

❤️C: 可用如下 4 种方法之一设置:

加法(包括比较指令 CMN)。若加法产生进位(即无符号溢出), 则 C 置 1, 否则置 0。

减法(包括比较指令 CMP)。若减法产生借位(即无符号溢出), 则 C 置 0, 否则置 1。

对于结合移位操作的非加法/减法指令, C 置为移出值的最后 1 位。

对于其他非加法/减法指令, C 通常不改变。

❤️V: 可用如下两种方法设置

对于加法或减法指令, 当发生带符号溢出时, V 置 1, 认为操作数和结果是补码形式的带符号整数。

对于非加法/减法指令, V 通常不改变。

3)控制位程序状态寄存器 CPSR 的最低 8 位 I、F、T 和 M[4: 0]用作控制位。当异常出现时改变控制位。处理器在特权模式下时也可由软件改变。

❤️中断禁止位: I 置 1, 则禁止 IRQ 中断, F 置 1, 则禁止 FIQ 中断

❤️T 位: T=0 指示 ARM 执行, T=1 指示 Thumb

❤️执行模式控制位 M4、M3、M2、M1 和 M0(M[4:0])是模式位, 决定处理器的工作模式, 具体含义见处理器的 7 种工作模式。

当前程序状态寄存器CPSR

功能

设置处理器运行模式和工作状态

控制允许或者禁止FIQ和IRQ

保存ALU当前操作信息，设置条件标志位，控制条件执行

SPSR

备份状态寄存器，当异常发生时，用于保存CPSR的状态。
usr和sys模式下无SPSR

⑥SPSR 被称为备份程序状态寄存器，共有五个，每个异常模式下都有一个。用于特定的异常中断发生时保存 CPSR 的内容，用户模式和系统模式没有 SPSR。

47.异常处理机制

(一) 异常处理模式

当异常发生时，处理器会把 PC 设置成一个特定的存储器地址，作为异常程序的入口地址，这个地址称为异常向量。异常向量指向一条跳转指令，从而指向具体的异常处理程序。异常处理模式如下：

异常类型	优先级	处理器模式	异常向量	说明
复位 RESET	1	管理模式	0x00000000 0	复位引脚有效时进入
未定义指令 UND	6	未定义模式	0x00000000 4	协处理器认为当前指令未定义时产生
软件中断 SWI	6	管理模式	0x00000000 8	用户定义的中断指令
指令预取中止 PABT	5	中止模式	0x00000000 C	预取指令目标不存在或该地址不允许当前指令访问
数据中止访问 DABT	2	中止模式	0x00000001 0	指令目标地址不存在或该地址不允许当前指令访问
IRQ 中断	4	外部中断	0x00000001 8	有外部中断时发生的异常
FIQ 中断	3	快速中断	0x00000001 C	有快速中断时发生的异常

48. 异常发生时 ARM 按照以下步骤响应

- (1) 把将来的返回地址保存到对应模式下的 R14 寄存器中。
- (2) 把 CPSR 的值保存到对应模式下的 SPSR 寄存器中。
- (3) 设置 CPSR 中相应的位。第 0-4 位为对应的处理器模式;第 5 位设置为 0,表示执行在 ARM 指令集状态;如果异常模式是复位或快速中断,则设置第 6 位为 1,表示禁止快速中断,否则不设置第 6 位;第 7 位设置为 1,表示禁止外部中断。
- (4) 设置 PC 为对应的异常向量。
- (5) 跳转至异常处理程序,处理异常
- (6) 异常的返回:
 - ①恢复原来被保护的寄存器。
 - ②将相应 SPSR 中的值复制到 CPSR 中。
 - ③根据异常类型将 PC 值恢复成断点地址,以继续执行用户原来运行的程序。

④清除 CPSR 中的中断禁止标志 I 和 F，开放外部中断 IRQ 和快速中断 FIQ。

2 异常处理机制（小结）

在异常被处理前,当前的处理器状态必须被保存,以便处理程序完成后,最后的程序可以被恢复.

**异常
响应**

拷贝 CPSR 到 SPSR_<mode>

保存返回地址到 LR_<mode>

设置控制位CPSR_c位
(ARM状态, 处理器模式, 设置中断)

设置 PC 为相应的异常向量 (即跳转地址)

**异常
退出**

从 SPSR_<mode> 恢复CPSR

从LR_<mode> 恢复PC

第七章 ARM 指令集

1. ARM 指令集的特点

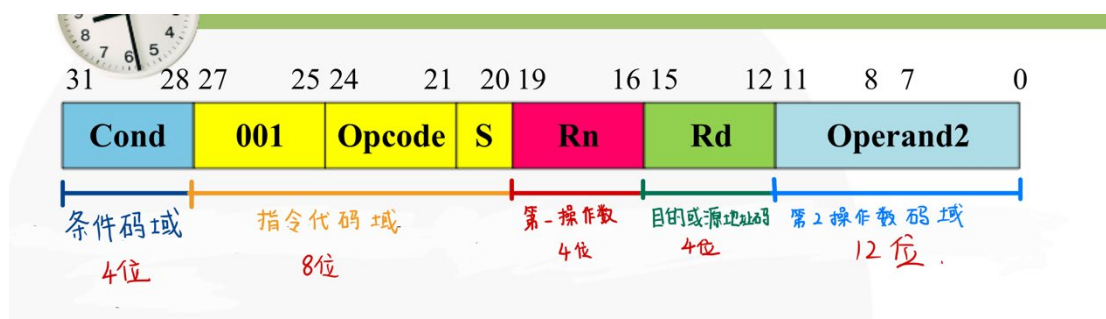
- 1) 使用大量的寄存器，所有数据处理都在寄存器中完成。
- 2) 只用 Load/Store 指令访问内存单元。也就是指令仅能处理寄存器的数据，处理结果仍要放回寄存器中，而对系统存储器的访问则需要通过专门的 Loader/Store 指令。外部存储器包含外围设备的寄存器、外部数据存储器或可访问的程序存储器等
- 3) 每条 ARM 指令都是 32 位固定长度
- 4) ARM 指令集对不同类型的指令设计了不同的寻址方式。（立即数寻址、寄存器寻址、寄存器间接寻址、

寄存器变址寻址、多寄存器寻址、块寻址，堆栈寻址和相对寻址在内的八种寻址方式，寻址方式编码在操作码中)

5) ARM 指令对指令中一个源操作数采取了以移位操作为基础的预处理操作。

6) ARM 指令格式中还增加了条件码。

2. ARM 指令格式



3. 寻址方式的概念

概念：处理器根据指令中给出的地址码字段来寻找操作数物理地址的方式。

目的：取得操作数。

4. 寻址方式种类

(1) 立即寻址:也叫立即数寻址，操作数直接包含在指令中，取出指令的同时也取到了操作数。

(2) 寄存器寻址:寄存器中的数值作为操作数。

(3) 寄存器间接寻址:以寄存器的值作为操作数的地址，而操作数本身存放在该地址指向的存储器中。

(4) 基址加变址寻址：以寄存器中的值加对应数值

形成操作数的地址，地址中的值为操作数。

(5) 多寄存器寻址：存储器中一块数据加载到多个寄存器，或者把多个寄存器中的内容保存到一片存储器区域。

(6) 相对寻址：以程序计数器 PC 的当前值为基地址，指令中的地址标号作为偏移量，将两者相加之后得到操作数的有效地址。

(7) 堆栈寻址：使用一个专门的寄存器(堆栈指针 SP)指向一块存储区域(堆栈)来指示当前的操作位置，堆栈总是指向栈顶的。

5. 立即数的预处理的原理

具体方法是把指令格式中，第 0~11 位分成两部分，其中 0~7 位用来存放一个 8 位的立即数 Imm-8，8~11 位存放一个 4 位的数值 rot，真正的立即数通过把 Imm-8 循环右移 $rot \times 2$ 次来生成，由于 rot 只有 4 位，最大表示值为 15，不能把 Imm-8 移位成 32 位，因此，ARM 规定循环右移次数为 $rot \times 2$ ，得到一个范围在 0-30，步长为 2 的移位值。

6. 四种类型的堆栈工作方式

(1) 满递增堆栈 FA: 堆栈指针指向最后压入的数据，且由低地址向高地址生长。

(2) 满递减堆栈 FD: 堆栈指针指向最后压入的数据，且由高

地址向低

地址生长。

(3)空递增堆栈 EA:堆栈指针指向下一个将要放入数据的空位置，且 由低地址向高地址生长。

(4)空递减堆栈 ED:堆栈指针指向下一个将要放入数据的空位置，且 由高地址向低地址生长。